



Detecção de Fraudes

Em cartões de crédito utilizando
SNN,LSTM e RL

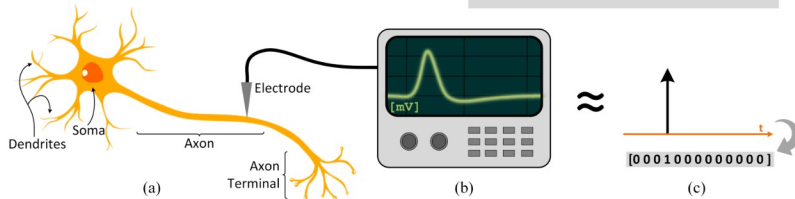
Santos, Antonio C. O.
23 July 2025

Fraud detection with SNN & LSTM

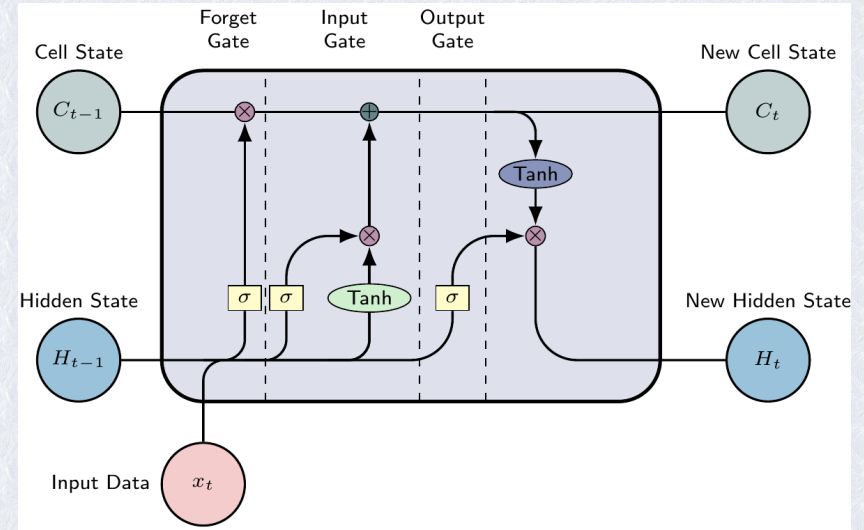
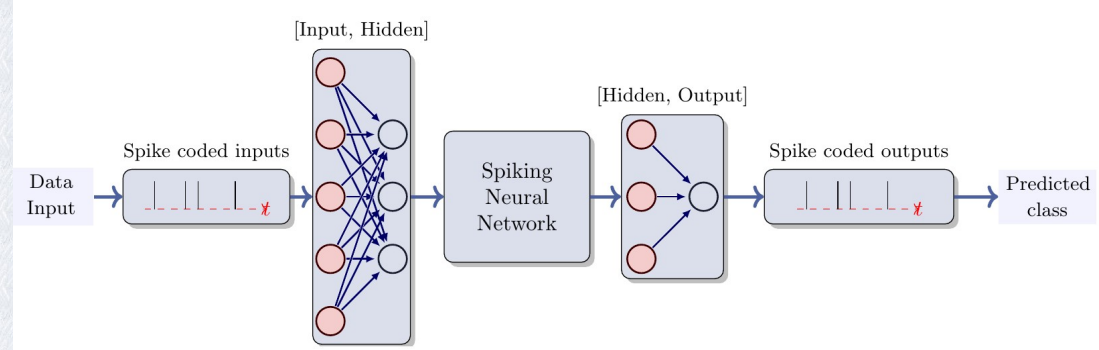
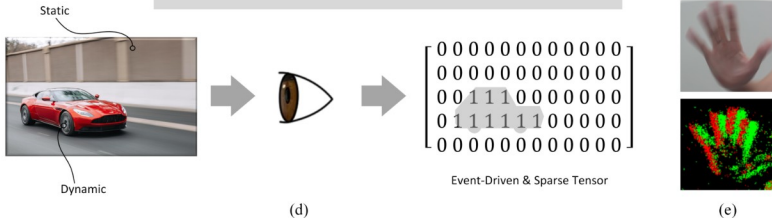
Spikes, Sparsity and Static-Suppression

Neurons communicate via action potentials, or **spikes**

Spikes are approximated as 1's and 0's, which can be efficiently represented in **sparse data structures**



In the absence of change, input data is **suppressed**. This enables an energy-efficient mode of data representation.

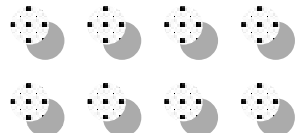


Sumário

- Objetivos
- Dados
- SNN
- LSTM
- SNN+LSTM
- Resultados (SNN,LSTM)
- Reinforcement Learning
- Conclusões parciais



Illustrations by Pixeltrue on
[icons8](#)



Objectives

- Implementation of a QML version from:

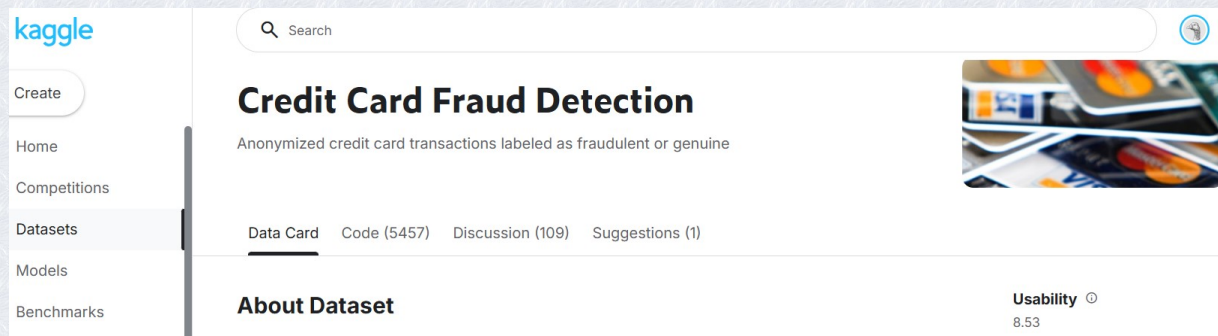
Brain-Inspired Quantum Neural Architectures (Arxiv:
[2505.01735](#)).

For Fraud detection (credit card).

Reobtaining old benchmark results ([2505.01735](#)) and others, like
XGBoost.

Dataset

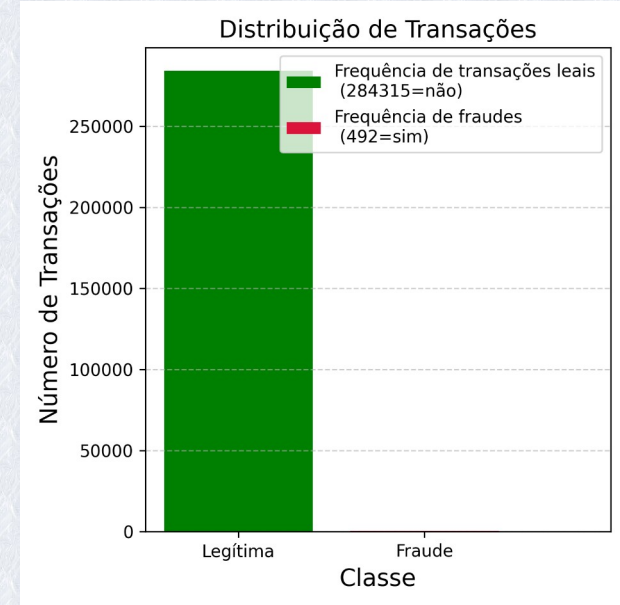
- Fraud detection (credit card)
 - The dataset: transactions made by credit cards in September 2013 by European cardholders.
 - This dataset presents transactions that occurred in two days, where we have 492 frauds out of 284,807 transactions. The dataset is highly unbalanced, the positive class (frauds) account for 0.172% of all transactions.



<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

Dataset and objectives

- Fraud detection (credit card)
 - This dataset presents transactions that occurred in two days, where we have 492 frauds out of 284,807 transactions. The dataset is highly unbalanced, the positive class (frauds) account for 0.172% of all transactions.
 - Número de dados (linhas): 284807
 - Número de colunas: 31
 - Colunas: ['Time', 'V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8', 'V9', 'V10', 'V11', 'V12', 'V13', 'V14', 'V15', 'V16', 'V17', 'V18', 'V19', 'V20', 'V21', 'V22', 'V23', 'V24', 'V25', 'V26', 'V27', 'V28', 'Amount', 'Class']



Packages version SNN & LSTM

- Python 3.11.13
- Jupyter-core 5.8.1
- Matplotlib 3.7.3
- Notebook 7.4.4
- Numpy 1.24.4
- pip 25.1
- Snntorch 0.9.4
- sympy 1.14.0
- torch 2.7.0
- torchvision 0.22.0

BRAIN-INSPIRED QUANTUM NEURAL ARCHITECTURES FOR PATTERN
RECOGNITION: INTEGRATING QSNN AND QLSTM

Eva Andrés, Manuel Pegalajar Cuéllar, Gabriel Navarro
Department of Computer Science and Artificial Intelligence
University of Granada
C/ Periodista Daniel Saucedo Aranda, s/n, 18071, Granada, Spain
e.evaandres@go.ugr.es, manupc@decsai.ugr.es, navarro@decsai.ugr.es

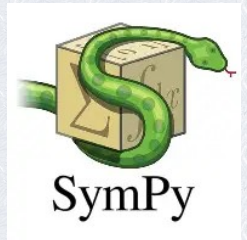
ArXiv: 2505.01735

The logo for snnTorch, featuring three vertical bars of increasing height (red, orange, yellow) to the left of the text "snnTorch".

snnTorch

The PyTorch logo, consisting of a red circular flame-like icon to the left of the text "PyTorch".

PyTorch



Packages version SNN & LSTM

- Python 3.11.13
- Jupyter-core 5.8.1
- Matplotlib 3.7.3
- Notebook 7.4.4
- Numpy 1.24.4
- pip 25.1
- Snntorch 0.9.4
- sympy 1.14.0
- torch 2.7.0
- torchvision 0.22.0

TRAINING SPIKING NEURAL NETWORKS USING LESSONS FROM
DEEP LEARNING

Jason K. Eshraghian*
UC Santa Cruz
University of Michigan
jeshragh@ucsc.edu

Max Ward
UWA
Harvard University

Emre Neftci
Forschungszentrum Jülich
RWTH Aachen

Xinxin Wang
University of Michigan

Gregor Lenz
SynSense

Girish Dwivedi
UWA

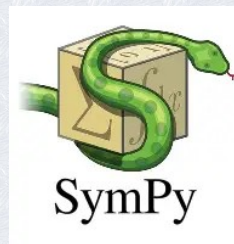
Mohammed Bannamoun
UWA

Doo Seok Jeong
Hanyang University

Wei D. Lu*
University of Michigan
wluee@umich.edu

 snnTorch

 PyTorch



ArXiv: 2109.12894v6

SNN

(Snntorch_0.9.4 e torch_2.7.0)

```
# ----- 2. Definição do modelo SNN -----  
class SNNModel(nn.Module):  
    def __init__(self, input_size, hidden_sizes, output_size, beta=0.9, dropout_prob=0.0, surrogate_type='fast_sigmoid'):  
        super().__init__()  
        self.layers = nn.ModuleList()  
        self.lifs = nn.ModuleList()  
        self.dropouts = nn.ModuleList()  
  
        # Escolher surrogate gradiente  
        if surrogate_type == 'fast_sigmoid':  
            grad_fn = surrogate.fast_sigmoid()  
        elif surrogate_type == 'piecewise_linear':  
            grad_fn = surrogate.piecewise()  
        else:  
            grad_fn = surrogate.fast_sigmoid() # padrão  
  
        last_size = input_size  
        for h in hidden_sizes:  
            self.layers.append(nn.Linear(last_size, h))  
            self.dropouts.append(nn.Dropout(dropout_prob))  
            self.lifs.append(snn.Leaky(beta=beta, spike_grad=grad_fn))  
            last_size = h  
  
        self.out = nn.Linear(last_size, output_size)  
        self.lif_out = snn.Leaky(beta=beta, spike_grad=grad_fn)  
  
    def forward(self, x, num_steps=25):  
        mems = [lif.init_leaky() for lif in self.lifs]  
        mem_out = self.lif_out.init_leaky()  
        spk_out_rec = []  
  
        for _ in range(num_steps):  
            cur = x  
            for i, (layer, dropout, lif) in enumerate(zip(self.layers, self.dropouts, self.lifs)):  
                cur = layer(cur)  
                cur = dropout(cur)  
                spk, mems[i] = lif(cur, mems[i])  
                cur = spk  
            cur = self.out(cur)  
            spk_out, mem_out = self.lif_out(cur, mem_out)  
            spk_out_rec.append(spk_out)  
  
        return torch.stack(spk_out_rec)
```

SNN

(Snntorch_0.9.4 e torch_2.7.0)

- HyperOpt:

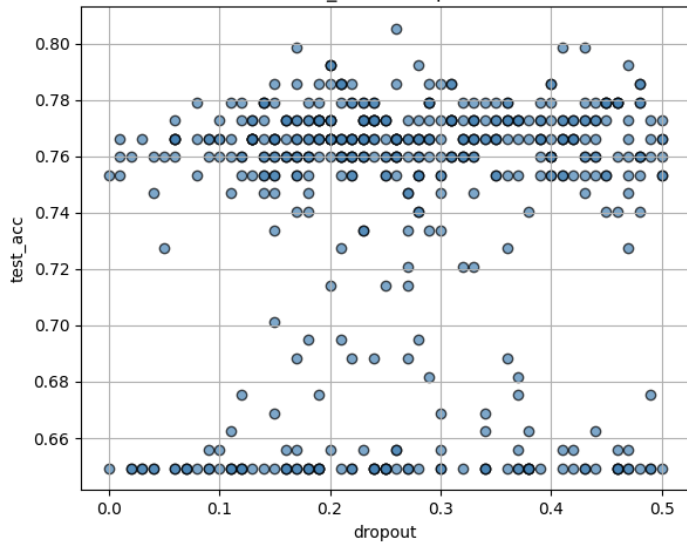
- <https://hyperopt.github.io/hyperopt/>
- <https://iopscience.iop.org/article/10.1088/1749-4699/8/1/014008>
- <https://hyperopt.github.io/hyperopt/>

```
# ----- 2. Definição do modelo SNN -----  
class SNNModel(nn.Module):  
    def __init__(self, input_size, hidden_sizes, output_size, beta=0.9, dropout_prob=0.0, surrogate_type='fast_sigmoid'):  
        super().__init__()  
        self.layers = nn.ModuleList()  
        self.lifs = nn.ModuleList()  
        self.dropouts = nn.ModuleList()  
  
        # Escolher surrogate gradiente  
        if surrogate_type == 'fast_sigmoid':  
            grad_fn = surrogate.fast_sigmoid()  
        elif surrogate_type == 'piecewise_linear':  
            grad_fn = surrogate.piecewise()  
        else:  
            grad_fn = surrogate.fast_sigmoid() # padrão  
  
        last_size = input_size  
        for h in hidden_sizes:  
            self.layers.append(nn.Linear(last_size, h))  
            self.dropouts.append(nn.Dropout(dropout_prob))  
            self.lifs.append(LIF(beta=beta, spike_grad=grad_fn))  
            last_size = h
```

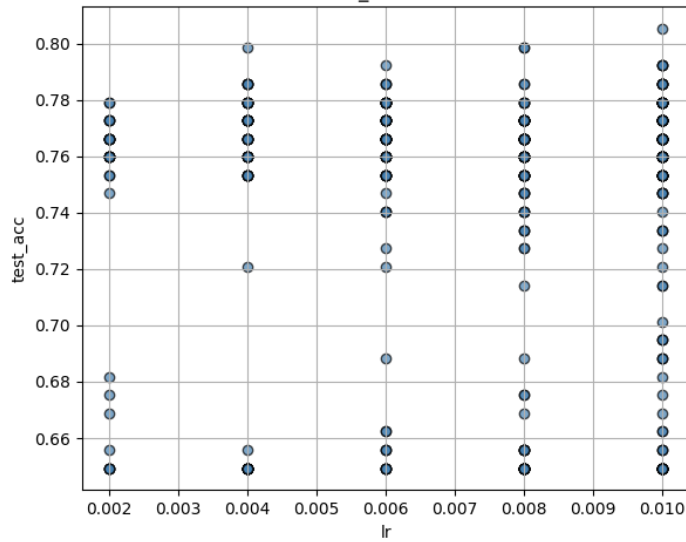
```
# ----- 4. Espaço de busca do Hyperopt -----  
space = {  
    'epochs': hp.quniform('epochs', 50, 150, 10),  
    'num_hidden_layers': hp.choice('num_hidden_layers', [1, 2, 3, 4]),  
    'neurons': hp.quniform('neurons', 64, 254, 16),  
    'lr': hp.quniform('lr', 1e-3, 1e-2, 2e-3),  
    'batch_size': hp.quniform('batch_size', 64, 128, 16),  
    'num_steps': hp.quniform('num_steps', 10, 100, 10),  
    'beta': hp.uniform('beta', 0.7, 0.99),  
    'dropout_prob': hp.uniform('dropout_prob', 0.0, 0.5),  
    'surrogate_type': hp.choice('surrogate_type', ['fast_sigmoid', 'atan']),  
    'optimizer_type': hp.choice('optimizer_type', ['adam', 'sgd'])  
}
```

SNN+Hyperopt

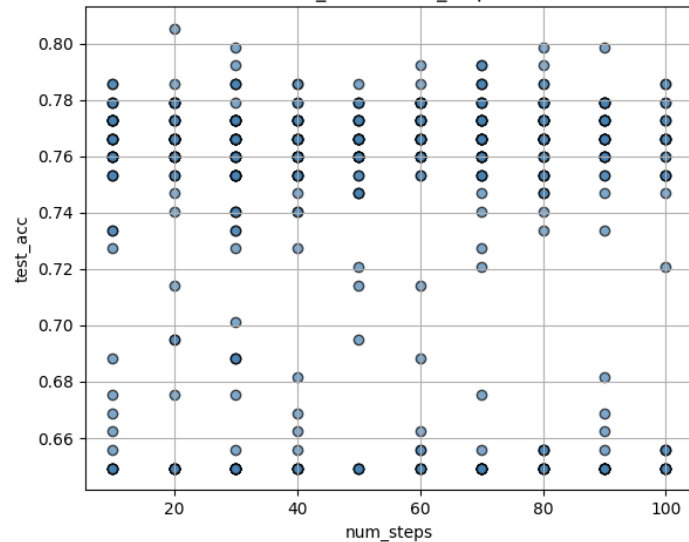
test_acc vs dropout



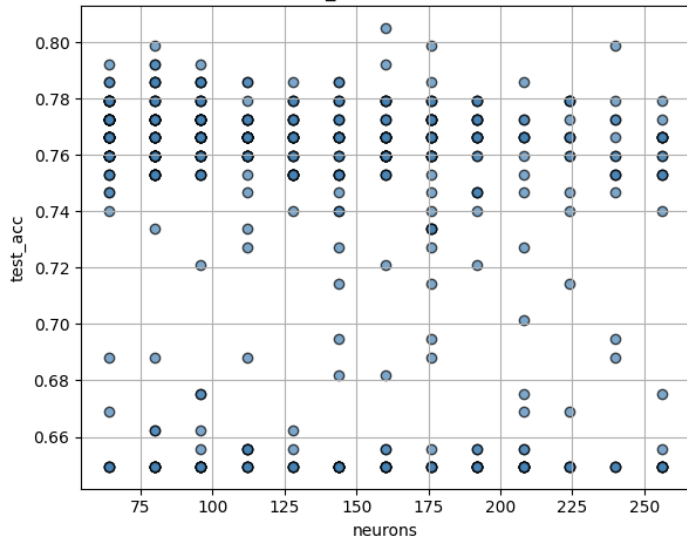
test_acc vs lr



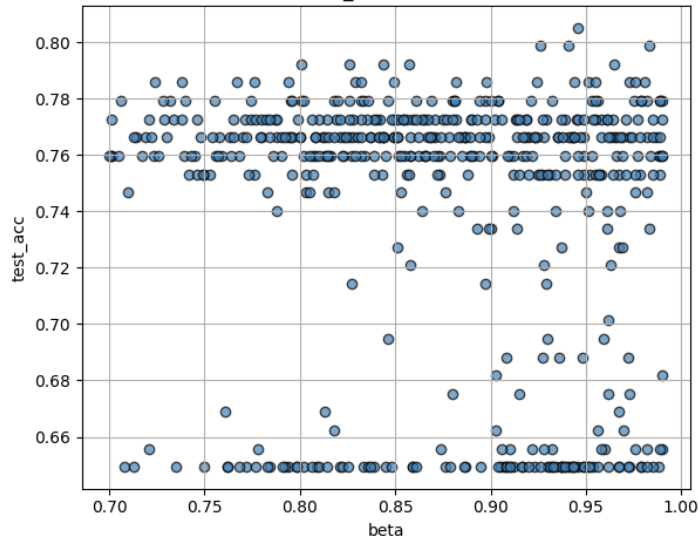
test_acc vs num_steps



test_acc vs neurons

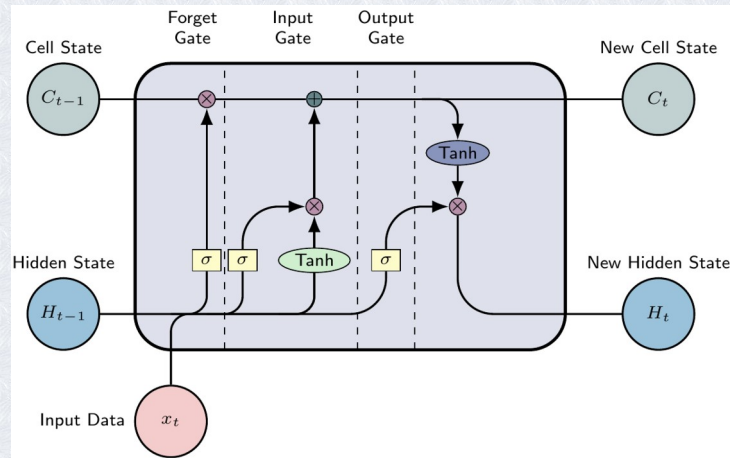


test_acc vs beta



SNN+Hyperopt

LSTM+Hyperopt



Toward Practical Quantum Machine Learning: A Novel Hybrid Quantum LSTM for Fraud Detection

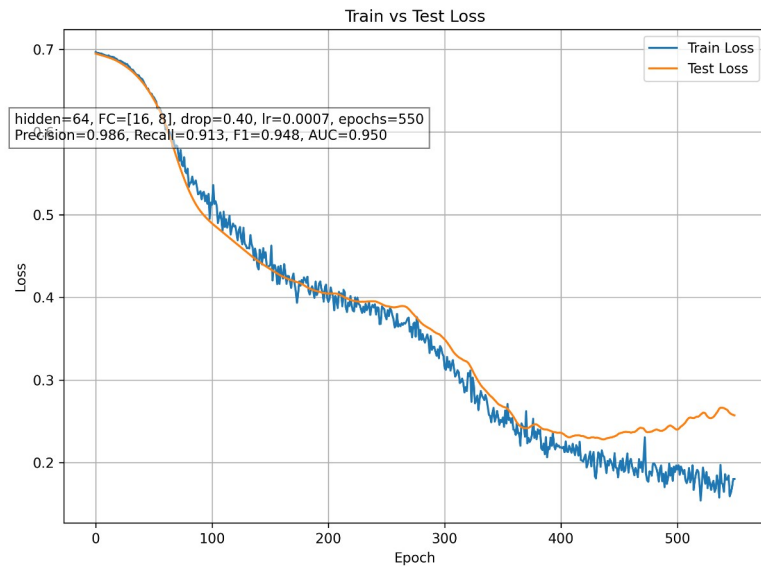
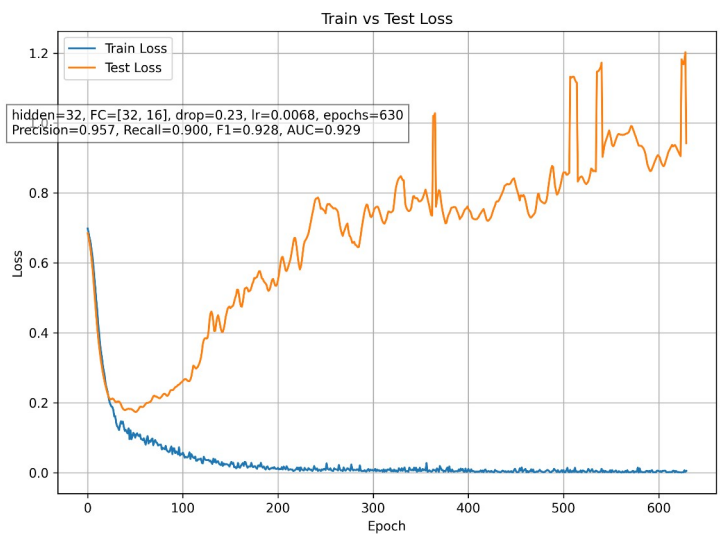
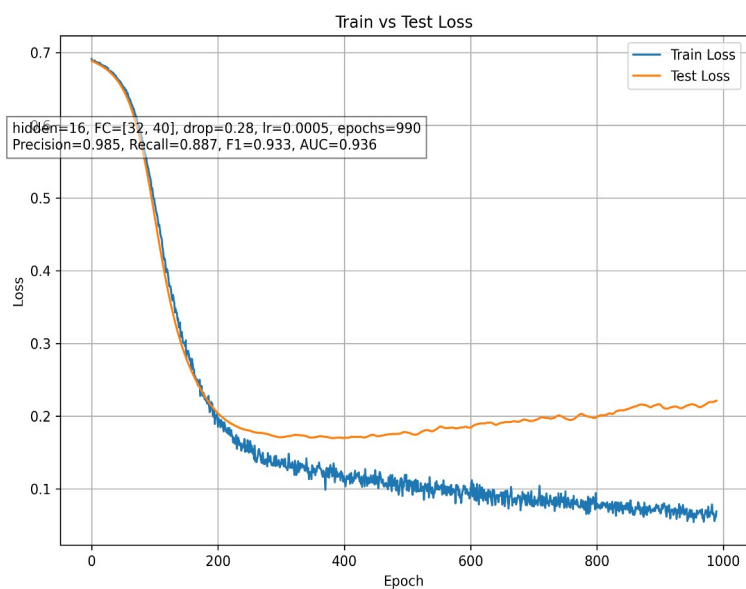
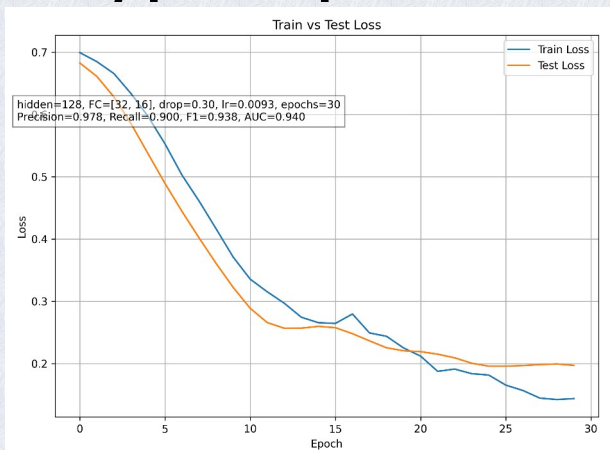
Rushikesh Ubale
Qkrishi Quantum
India
rushikesh.ubale@qkrishi.com

Sujan K. K.
Theoretical Sciences Unit
Jawaharlal Nehru Centre For
Advanced Scientific Research, India
sujanbharadwaj97@gmail.com

Sangram Deshpande
Dept. of ECE
NC State University
Raleigh, NC, USA
ssdesh24@ncsu.edu

Gregory T. Byrd
Dept. of ECE
NC State University
Raleigh, NC, USA
gbyrd@ncsu.edu

LSTM+Hyperopt

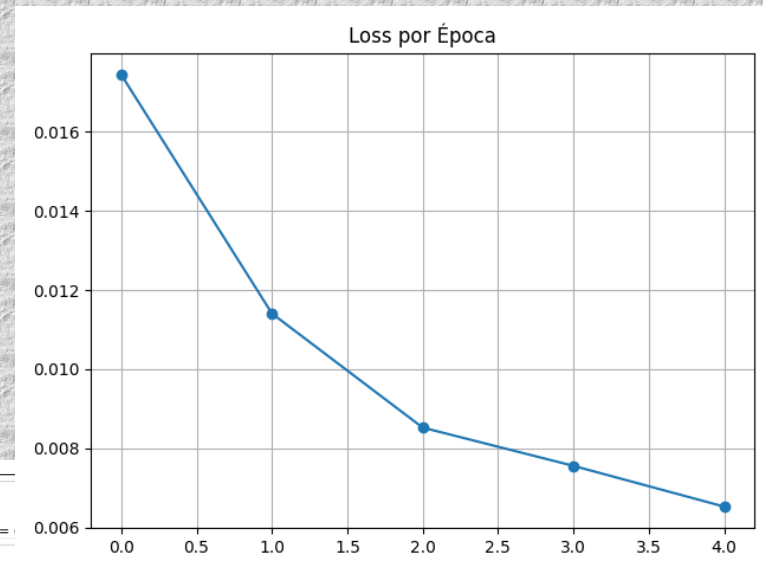
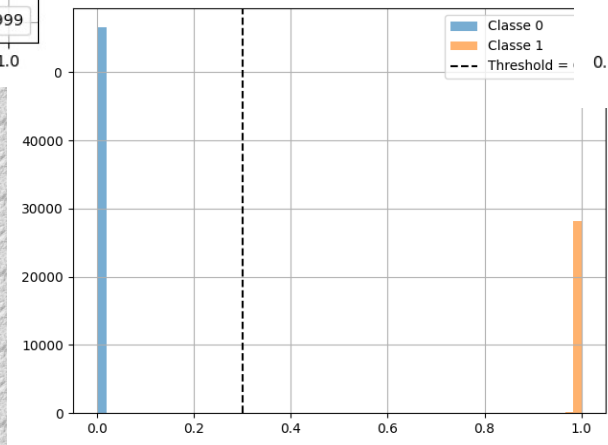
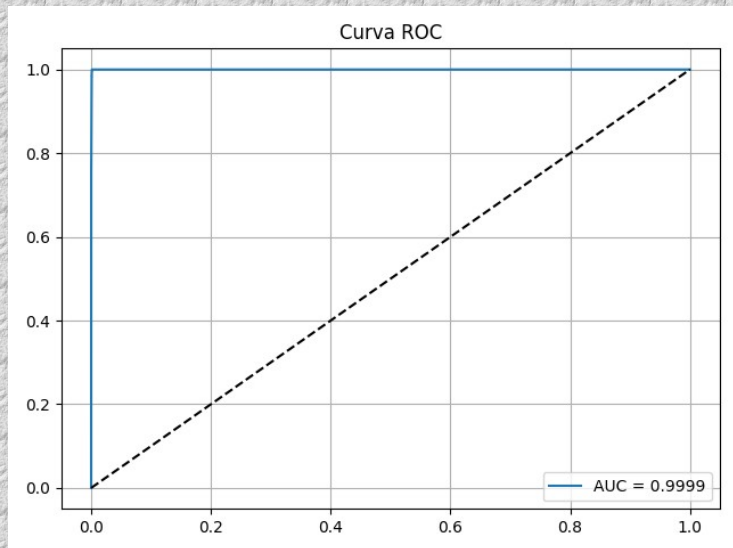


SNN + LSTM

SNN + LSTM

```
1
2 # 2. Modelo
3 print ('Model Load')
4 class SNNLSTMNet(nn.Module):
5     def __init__(self, input_size, lstm_hidden=64, fc1=16, fc2=16, dropout=0.3):
6         super().__init__()
7         self.lstm = nn.LSTM(input_size, lstm_hidden, batch_first=True)
8         self.lif = snn.Leaky(beta=0.9)
9         self.fc1 = nn.Linear(lstm_hidden, fc1)
10        self.dropout = nn.Dropout(dropout)
11        self.fc2 = nn.Linear(fc1, fc2)
12        self.out = nn.Linear(fc2, 1)
13
```

SNN + LSTM



SNN & LSTM – Próximas Análises

- Compreender melhor todas as etapas dos modelos e códigos.
- Buscar erros nos códigos (por exemplo se estamos dividindo corretamente o train/test).
- Ajustar os códigos aos artigos:
 - LSTM (Toward...), e QSNN+QLSTM (Brain inspired).
 - Iniciar o QML.
- 2505.00137v1 Toward Practical Quantum Machine Learning: A Novel Hybrid Quantum LSTM for Fraud Detection
- 2505.01735v1 Brain-Inspired Quantum Neural Architectures for Pattern Recognition: Integrating QSNN and QLSTM

Theoretical References

- 2505.01735v1 Brain-Inspired Quantum Neural
- 2109.12894v6 TRAINING SPIKING NEURAL NETWORKS USING LESSONS FROM DEEP LEARNING
- Biological neuron mode (wiki):
https://en.wikipedia.org/wiki/Biological_neuron_model#Leaky_integrate-and-fire
- <https://snntorch.readthedocs.io/en/latest/tutorials/index.html>

Slides extras

First test:

- Version 1 (23.87 kB)
- diabetes.csv
 - Sumário
 - 1 arquivo
 - 9 columns
 - Inteiros 7
 - Decimais 2
- Atributos: 'Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'.

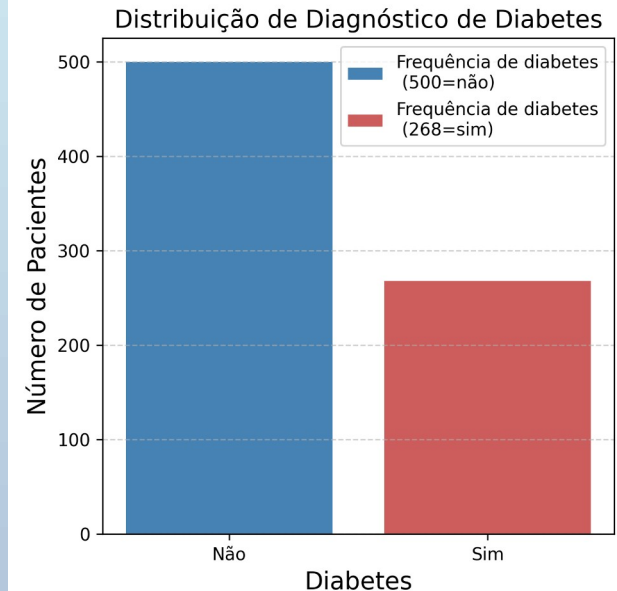
kaggle

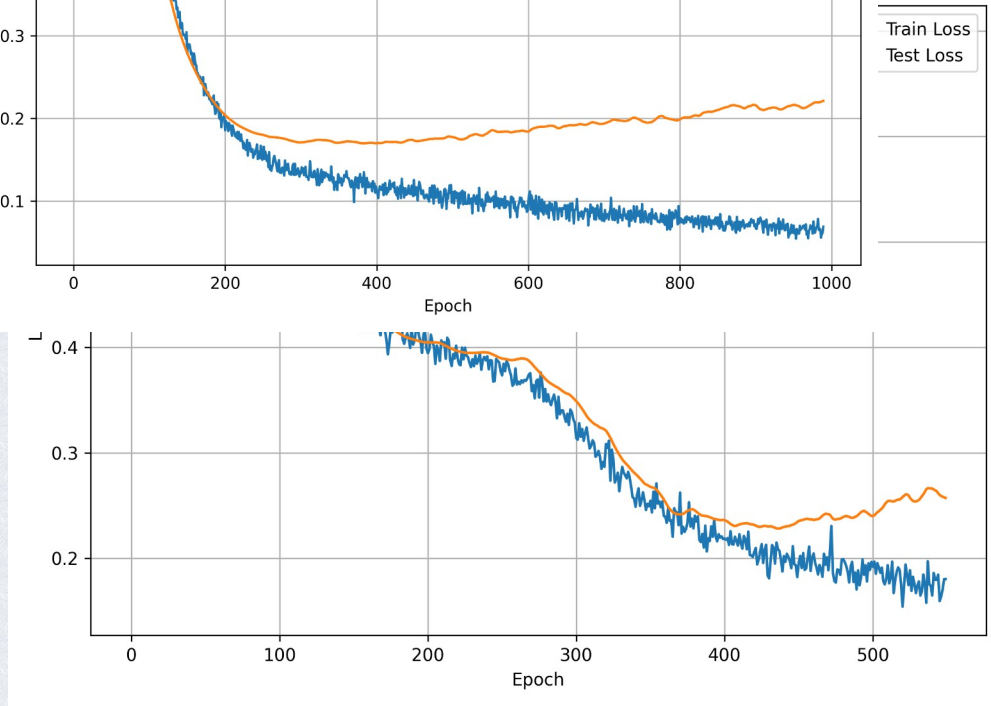
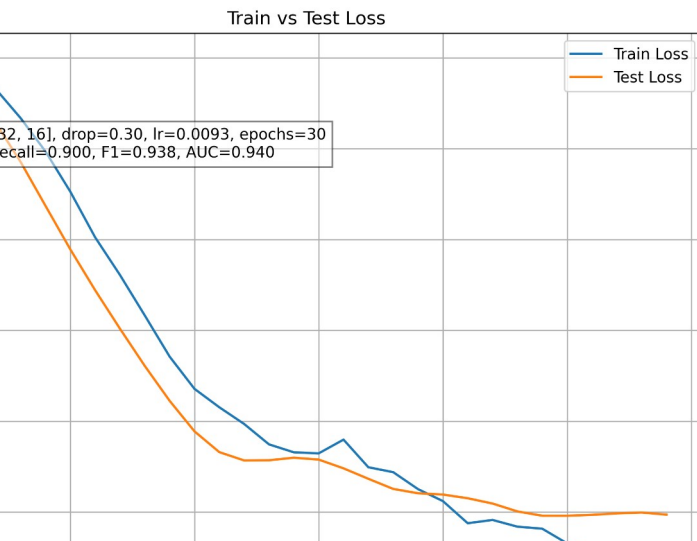
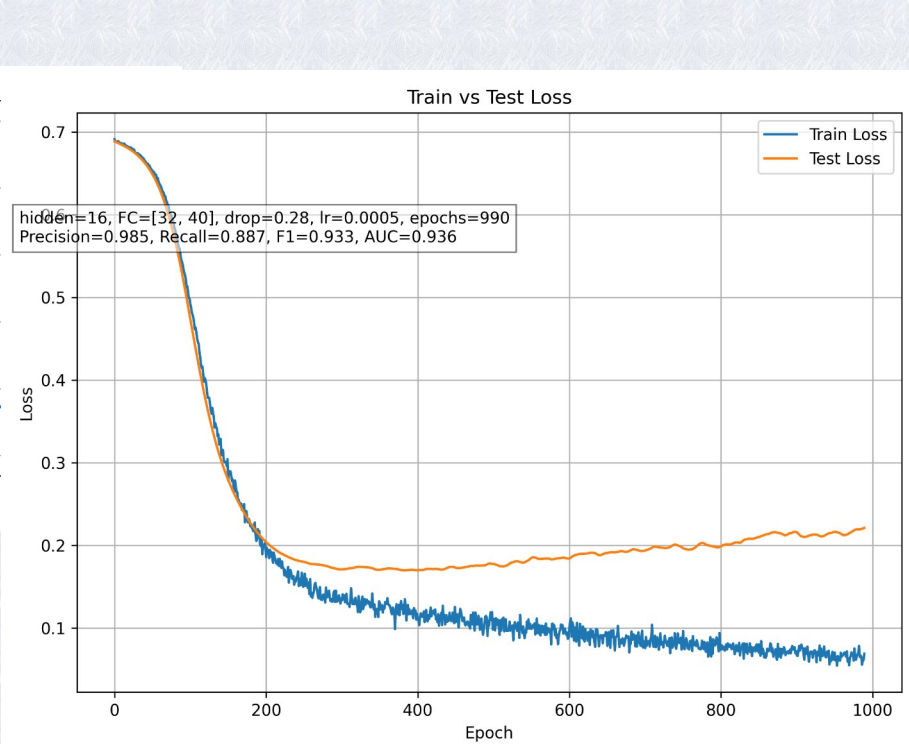
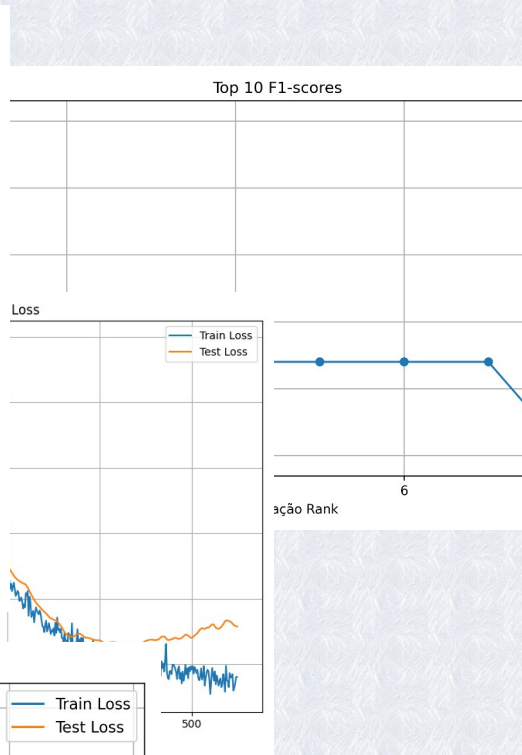
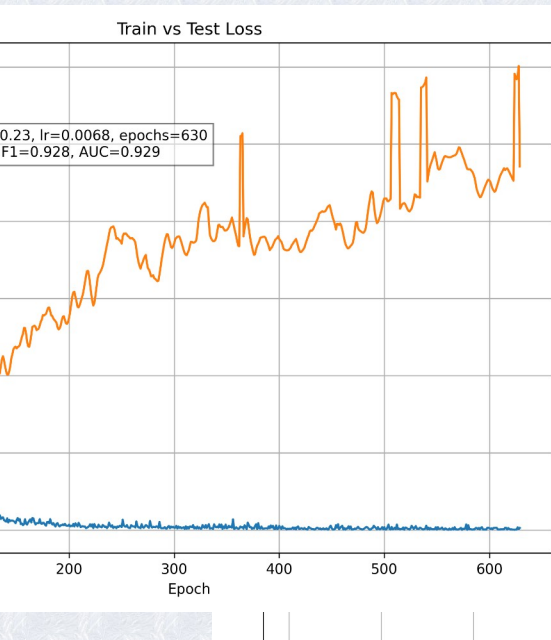
UCI MACHINE LEARNING AND 1 COLLABORATOR · UPDATED 9 YEARS AGO 4679

Pima Indians Diabetes Database

Predict the onset of diabetes based on diagnostic measures

Data Card Code (3588) Discussion (54) Suggestions (0)





LSTM+Hyperopt

